

1A) fsck() is the utility program that will be run. fsck() scans the metadata and structure of the filesystem to ensure consistency and is not corrupted after a crash. fsck() does this through recursive descent exploration of the pathname tree, visiting each directory and file entry and crosschecking that view with the view represented by the inode table and free block map. fsck() checks inconsistencies in the file system such as: blocks or fragments allocated to multiple files, inodes containing block numbers that overlap or are illegal (such as those marked free in the disk map). For instance, during this sudden power outage, fsck() may find that certain blocks or fragments are allocated to two files. In that case, fsck() repairs this by keeping the block with one file and then reassigning the conflicting inode to another block, or placing this inode into the lost+found directory so that the user can manually recover it.

ii) Mounting in a non-cached manner would mean that all completed writes would go directly to disk which reduces the risk of having large amounts of buffered data being lost in the event of a power outage. However, there is still the possible error of the partial writes. This means we could have inodes with missing or corrupted data fields, which would still require running fsck() to detect and possibly repair these inconsistencies.

1B) One reason would be the fact that about 5-10% of the disk space is reserved for system processes. Meaning that even with around 800GB of storage left, it would appear as “disk full” to a non-root user and not let you download more videos. Another reason would be the system overhead for each video file. Since the disk is formatted as EXT4 volume, each file requires extent descriptors and to describe how its blocks are stored. If the files are fragmented, multiple extent descriptors (which would be organized in extent trees) for each video file may be needed to map them which takes up disk space.

Additionally a video file may partially fill a block, but since the filesystems are allocated in fixed 4 KB blocks, the remaining storage would be wasted.

1C) A possible reason would be that /A/Z/ is on the same filesystem as /A/B whereas /A/C and /A/B are. Within the same filesystem, mv() becomes rename() which is very fast, but in the case from one filesystem to another, mv() needs to do a complete copy and rewrite which takes much more time for large files.

1D) st_size = 4, st_nlink = 1
st_mode = regular file, permissions: (0644/-rw-r- -r- -)
st_atime = 2025-09-24 16:55:00

st_mtime = 2025-09-18 22:30:10

st_ctime = 2025-09-24 16:30:00

2)

Data Block 42: 0-2047 (Contains Buffer A + Buffer B)
Data Block 43: (Contains Buffer C)
Data Block 44: (Contains Buffer C)
Data Block 45: (Contains Buffer C)
Data Block 46: (Contains Buffer C)
Data Block 47: (Contains Buffer C)
Data Block 48: (Contains Buffer C)
Data Block 49: (Contains Buffer C)
Data Block 50: (Contains Buffer C)
Data Block 51: (Contains Buffer C)
Data Block 52: (Contains Buffer C)
Data Block 53: (Contains Buffer C)
Single Indirect (On next page)
Double Indirect (On next page)
Triple Indirect (*UNUSED)

Single Indirect Block

Block 0: (EMPTY)
Block 1: 0-512 (Contains Buffer D)

Double Indirect Block

Block 0: [Block 0: 0-3 (the 3 bytes written are “ZZZ”)]

$$\text{st_block} = 17 * 4096 / 512 = 136$$

17 blocks = 12 direct blocks used + 2 indirect blocks used + 1 block referenced from Single Indirect + 1 single indirect block referenced from Double indirect + 1 block from Double indirect's referenced Single indirect block

$$\text{st_size} = 0x40C126 = 0x40C123 + 2 + 1 \text{ (+2 since the 3 bytes start writing at 0x40C123)}$$